

Computer Graphics: Lecture 7

Textures and Bind Groups

Slide Deck © Grant Williams, 2026, License: [CC-BY-SA 4.0](https://creativecommons.org/licenses/by-sa/4.0/)

Welcome Back!

Last time we learned all about color blending and vertex attributes.

We learned that every vertex can have multiple attributes attached to it, and that this data is passed to the vertex shader, one per `@location`.

We learned that we can interleave attributes by putting each attribute in some order, one per vertex.

We learned that the stride of a vertex buffer is how much to add to get from one vertex to another. This is the size of a vertex if attributes are interleaved. If we split attributes into different buffers, this will be the length of one “row”.

This time

We're finally going to learn about textures.

This is an important topic. Textures are the cornerstone of 3D graphics. Not only do they give objects their detail, they can be used as lookup tables for all kinds of effects (shadows, normal maps, reflections, the sky, many more things)

Furthermore, when we render a frame, we're actually drawing into a texture (which is why I've been handwaving `context.getCurrentTexture()`).

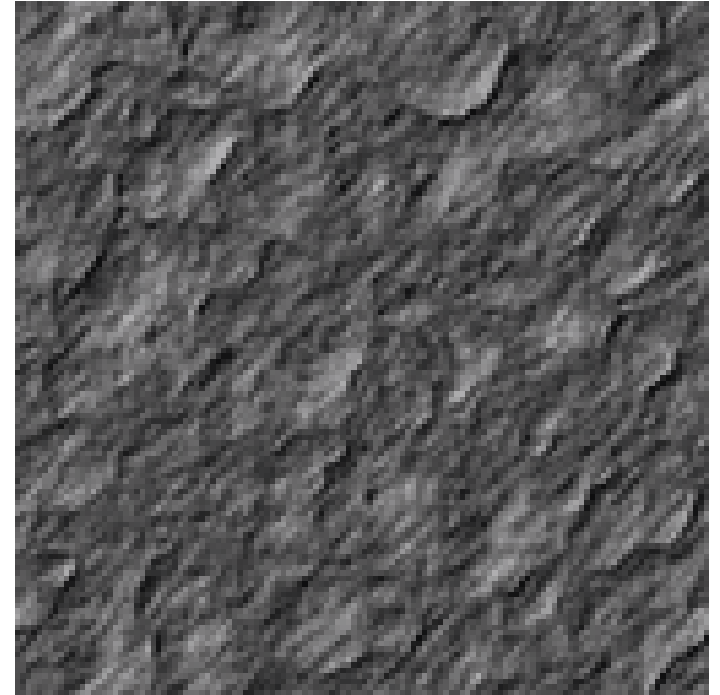
What is a texture?

Textures are images that we project on 3D shapes

But this is a bit oversimplified.

In fact, they have a complex internal storage, with multiple levels of images of different sizes.

We will learn the details in the advanced texturing later in the course.



A texture. (by [GGBot](#)—scaled up beyond its intended dimensions)

Why textures?

Textures actually serve a lot of purposes.

Our first use of them will be to give detail to 3D objects.

Since we haven't learned about perspective yet (it's coming, I promise!), textures will be a way to draw pixels on the screen instead of just colored gradients.

They can also be lookup tables, temporary storage, palettes, depth buffers, and even define shadows. They are extremely versatile.

What textures?

Textures can be 1-dimensional, 2-dimensional, or 3-dimensional. We will discuss 2-dimensional textures mainly in this course as those are the most commonly used.¹

In the same way that a normal image is composed of *pixels*, textures are composed of **texels**.

A **texel** is a *TEXTure ELement*.

Why a different term? Because like I mentioned before, textures actually have multiple levels. A texel is a pixel in one of the levels.

¹1D textures are useful for gradients and borders, 3D textures are useful for volumetric fog and medical imaging.

How textures?

In 3D textures are not just images we draw “at an angle”.

Instead, we draw a triangle, but we **sample** the texture as we do so.

Sampling a texture is like looking up a pixel in an image, but it is smooth. We can smoothly look up a point in between several texels.

We do this for every fragment on the triangle.

The key is knowing specifically which texels to look up at which point on the triangle.

Let's get concrete

Textures are a little weird, but once you understand how they work, they gradually become intuitive.

To make them intuitive, we're going to focus on one simple problem: drawing a textured quad.

That is, we're going to draw a rectangle, but we'll modify our fragment shader to look up the texels instead of blending a color.

UV coordinates

Textures are sampled using a special coordinate system. Understanding this coordinate system is critical to getting good at textures.

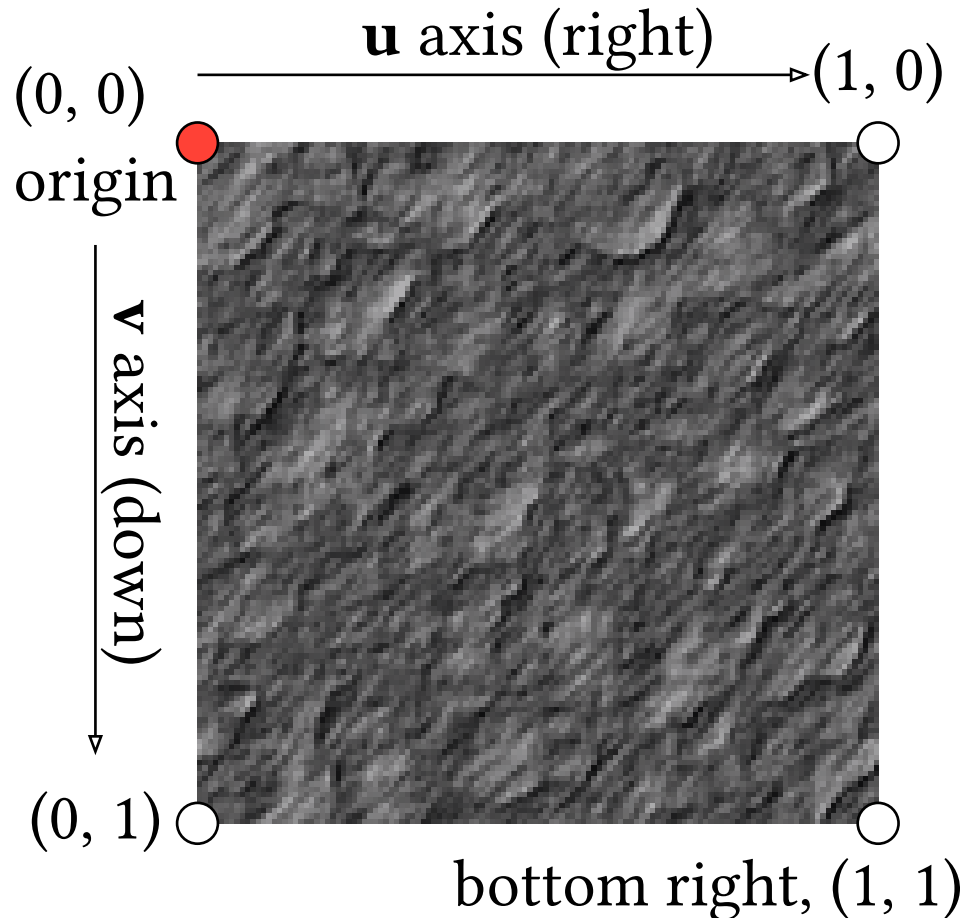
The coordinate system is called **uv** coordinates.

u and **v** are dimensions, like xy or rgb.

They are a way to describe the position of a sample on a texture, regardless of how big the texture is.

Let's see how they work...

Example UVs



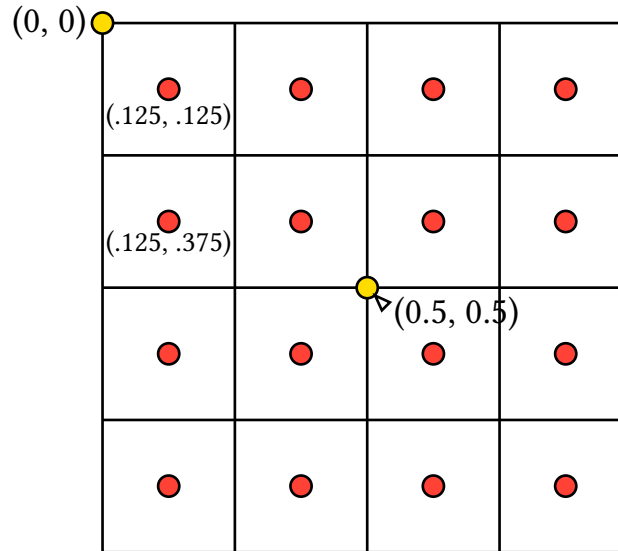
The u, v coordinate system treats the *origin* as the top left.

“ u ” and “ v ” refer to far to go down and right.

Our goal is to choose a point to “sample”, meaning, to retrieve the color at.

If we choose $(0.5, 0.5)$, we are sampling the very center of the image.

Here is a 4-by-4 texture with UV coordinates



Each cell is .25 wide and tall. The center (0.5, 0.5) is between 4 texels. If we sample that point, the GPU could give us one of those 4 colors, or it could average them together depending on our sampling mode.

With me so far?

Textures are like images, with *texels* instead of *pixels*.

The main things we do to textures is *sample* them, which means to look up a color from a position on the texture.

We don't provide the raw coordinates of the texel, instead we use the u/v coordinate system.

We do this so that we could replace the texture with a higher-resolution one (more pixels) and have the addressing still return roughly the same colors.

With me so far? (2)

U/V coordinates have $(0, 0)$ at the top left. The U coordinate points right, the V coordinate points down.

$(0.5, 0.5)$ is the *center* of the texture. For a power of 2 sized texture it refers to the center of a square of 4 texels.

$(0, 0)$ refers to the top left corner of the top left texel.

To get the actual center of the texel, you have to add an offset, which is half the width of a texel in u/v coordinates.

$1/\text{width}$ is the width of a texel in u coordinates. Half of that is the distance from the left border of a texel to its center. Likewise for V.

Questions?

Texture mapping a triangle

The act of applying a texture to something is called *texture mapping*.

Many texturing techniques have “mapping” in their name: normal mapping, shadow mapping, parallax mapping, etc.

What we want to do is color mapping. Have the *color* of a fragment come from a texture, instead of being blended from vertex output colors like we did last time.

Texture mapping a triangle (2)

Previously, we took advantage of the fact that colors were *interpolated* before the hit the fragment shader.

That is, if one vertex of a triangle was red, one was green, and one was blue, the fragment shader would receive the correct blend of red, green, and blue depending on how close it was to each vertex.

We want to do *the same thing* but with u/v coordinates instead of colors.

If we do that, we'll end up “blending” the u/v coordinates, so that nearby fragments sample nearby texels.

Texture mapping example

To understand this, let's imagine a single triangle.

This triangle will have a u/v attribute, which stores the u/v coordinates at each of its three vertices.

Each fragment on the surface of the triangle will interpolate between these three u/v values.

Inside the fragment shader, when the fragment samples the texture, it will pull a color that corresponds to the unique u/v coordinates of that fragment.

Texture mapping example (2)

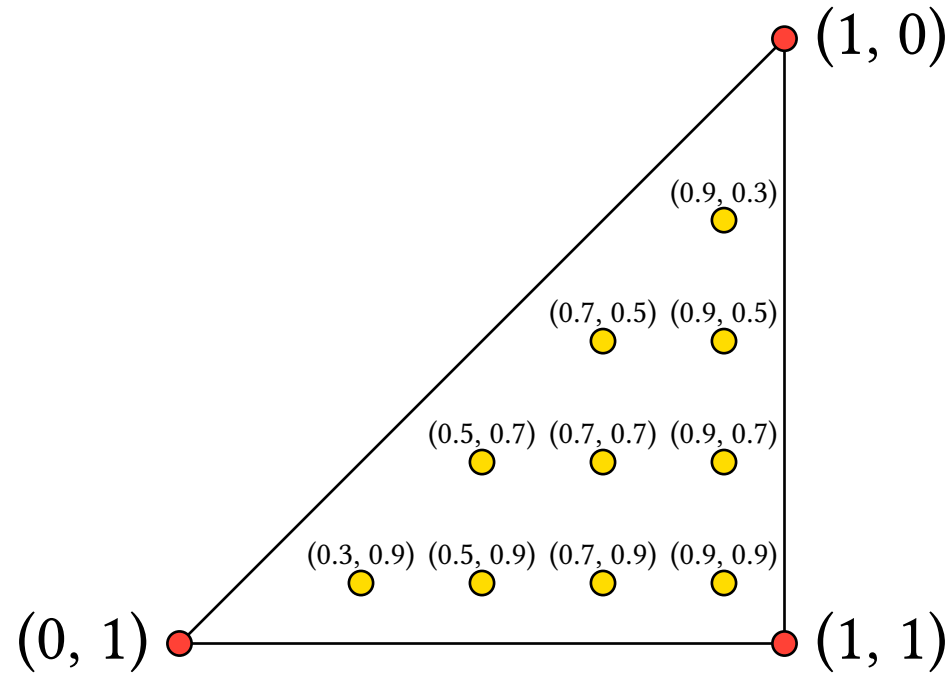
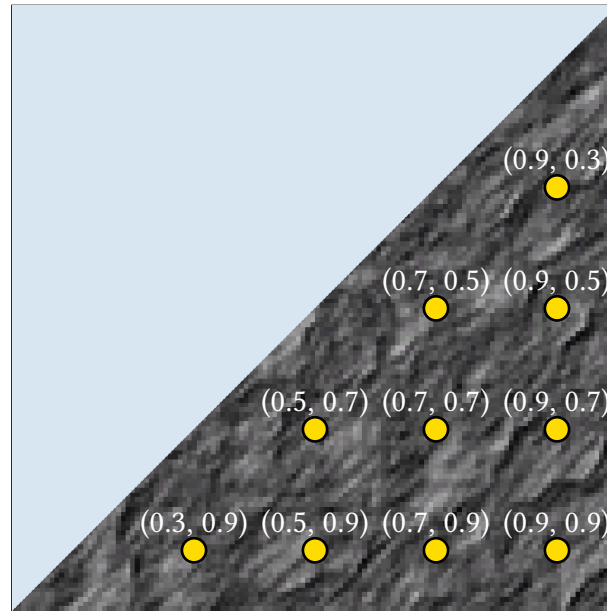


Figure 2: Each fragment (pixel within triangle) will receive an interpolated u/v coordinate. Therefore, the color looked up from the texture will be different for each one.

Texture mapping example (3)



Each fragment gets a uv coordinate that is interpolated between the uv coordinates of the corners. This happens for *every fragment*, but I'm only showing a few. (Imagine that there is a dot for every pixel on the screen)

Comprehension check

What would happen visually if we continued to sample $(1, 1)$ for the bottom left point, but we sampled $(1, .75)$ for the top right point, and $(.75, 1)$ for the bottom left point, but the triangle is the same size.

What would it look like?

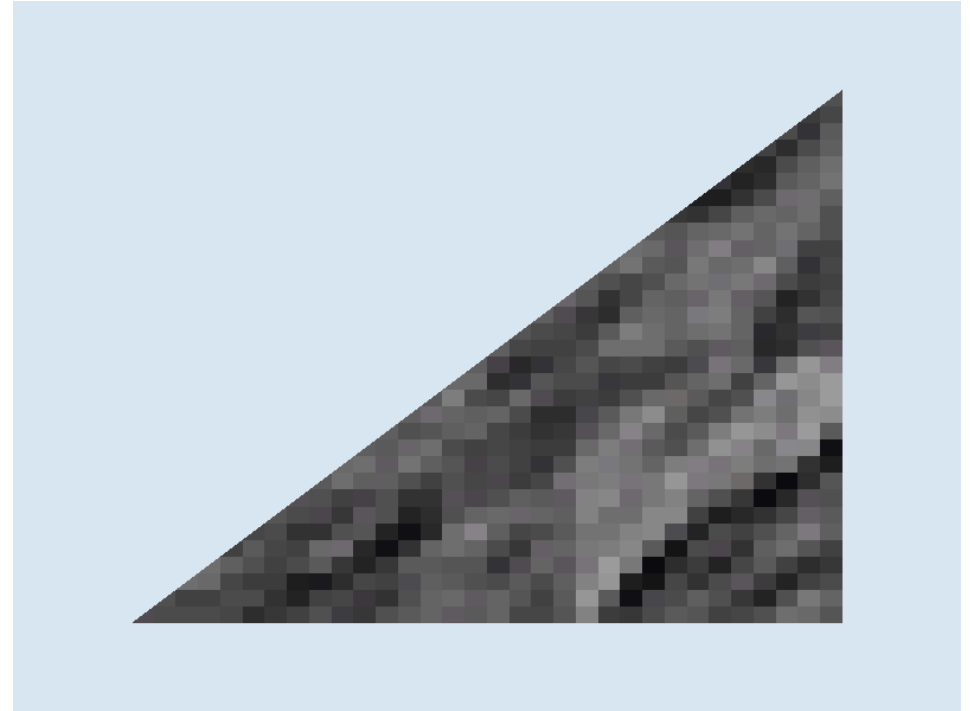
[Take a second and think about it]...

It would zoom in on the corner

We sample a smaller portion of the texture (the lower right sixteenth of it), but we are blowing it up over the same size of rectangle.

The number of pixels we're drawing stays the same, but the number of *texels* we're sampling is 1/16th as many.

Therefore, the texture is zoomed in, and looks grainy.¹



¹this is why games from the 5th generation of game consoles (i.e., the PS1, N64 generation) look so grainy/blurry. The textures had to be small due to hardware limitations.

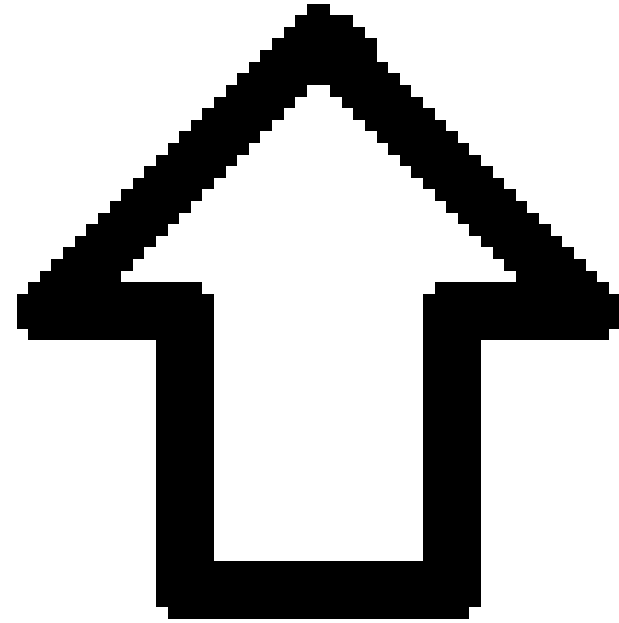
Questions?

Let's get artistic

Now that we understand a bit more about UV coordinates, let's try to think artistically about them.

That is, instead of imaging *what* it will look like, let's try imagining how to achieve an effect that we want. This is a question we will often find ourselves asking.

Suppose we have a texture that looks like an up arrow.



This is a 64 by 64 texture, so we can see the texels.

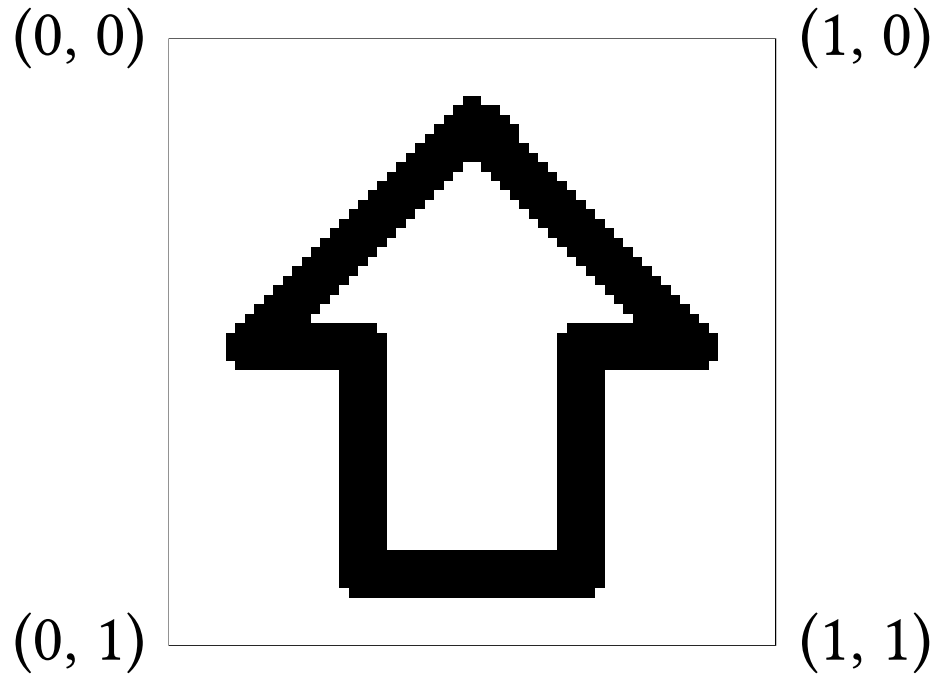
Our first goal

Let's just draw the arrow, but instead of a triangle, let's draw it to a quad, instead.

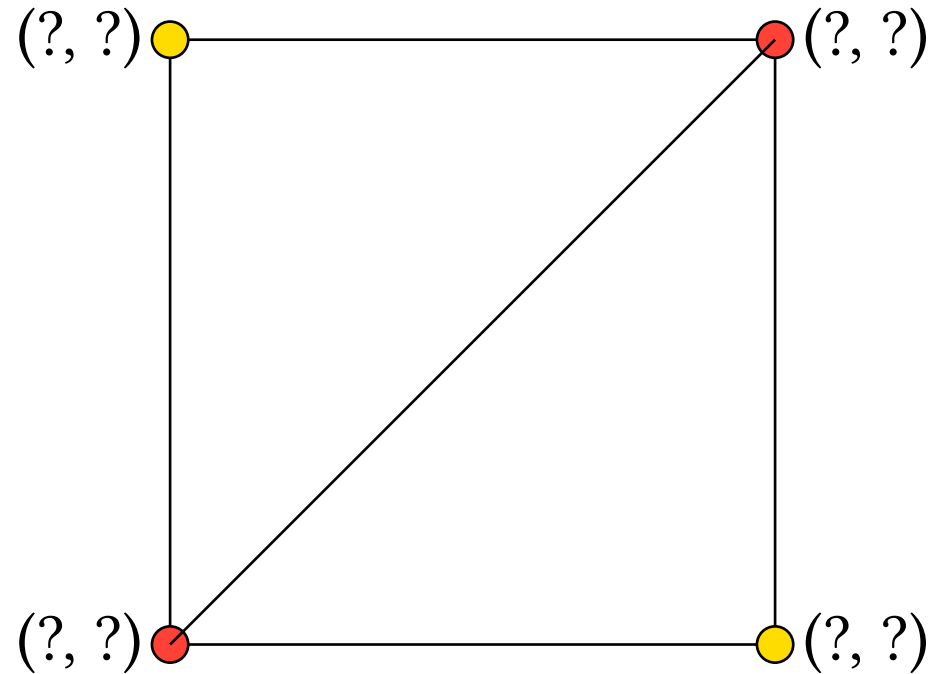
That means we will need to specify 6 vertices¹, two triangles.

¹There's a way to specify a quad with 4 verts using a different topology, but we haven't covered those yet.

What are the U/V coordinates for each vertex?

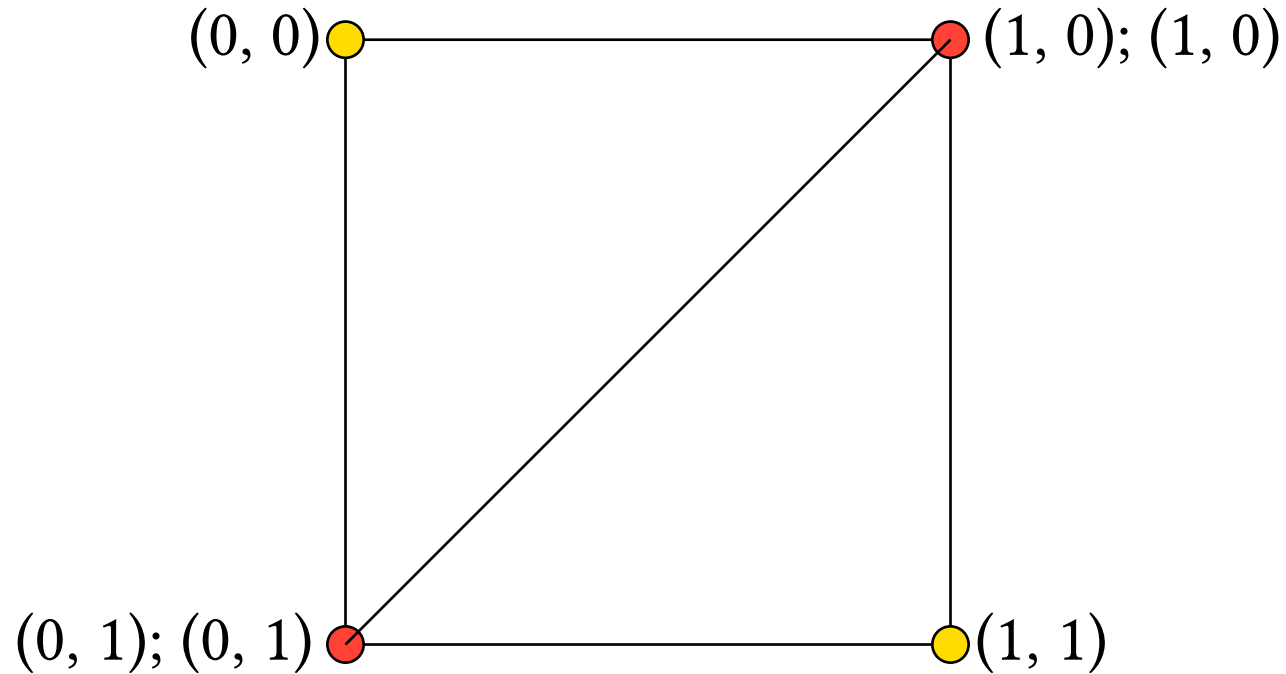


The texture, with the corner UV coordinates labelled. These are always the same for every texture.



The top right and bottom left points are duplicated between both triangles, but we want them to have the same U/V coordinates (otherwise there'd be a seam)

U/V coordinate answer



It's the same. We map the top left of the texture to the top left of the quad. We map the top right of the texture to both vertices in the top right. Ditto for bottom left. We map bottom-right to bottom-right.^{23 / 28}

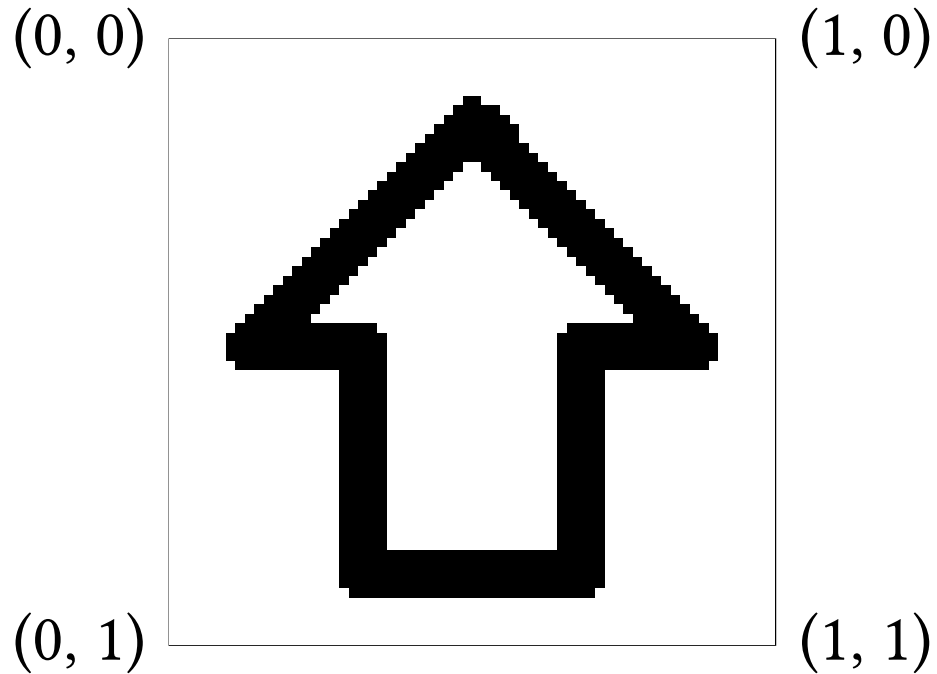
The principle

The basic idea when answering this question is: “what part of the texture do I want to be at this point on the triangle.”

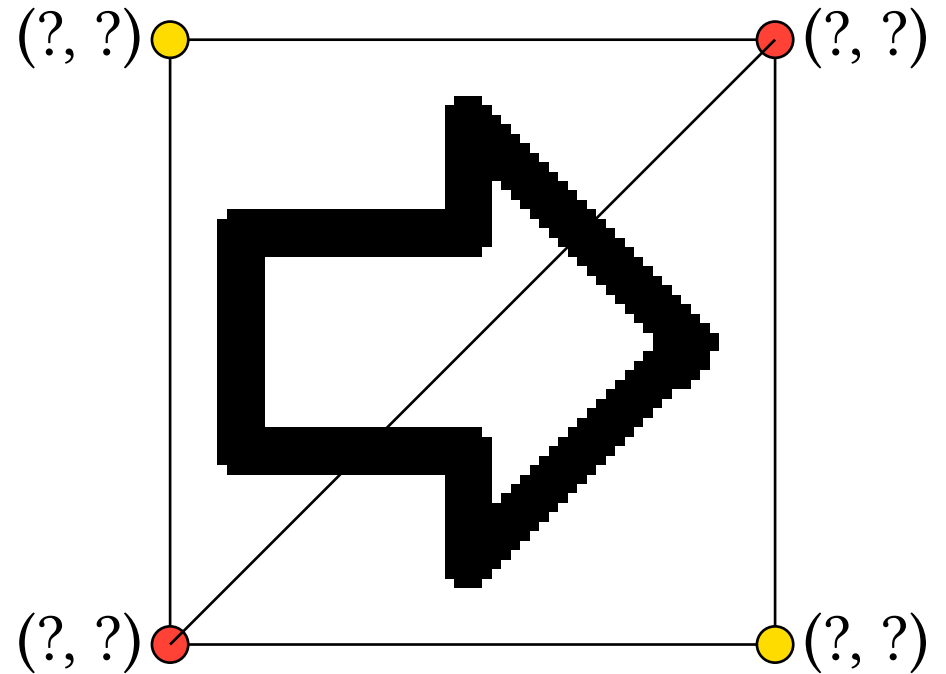
So let’s make it a bit harder. How do we map the triangle *sideways*?

Specifically, we want the arrow to be pointing to the right. This means we need a new batch of U/V coordinates. Which ones?

Right arrow mapping question?

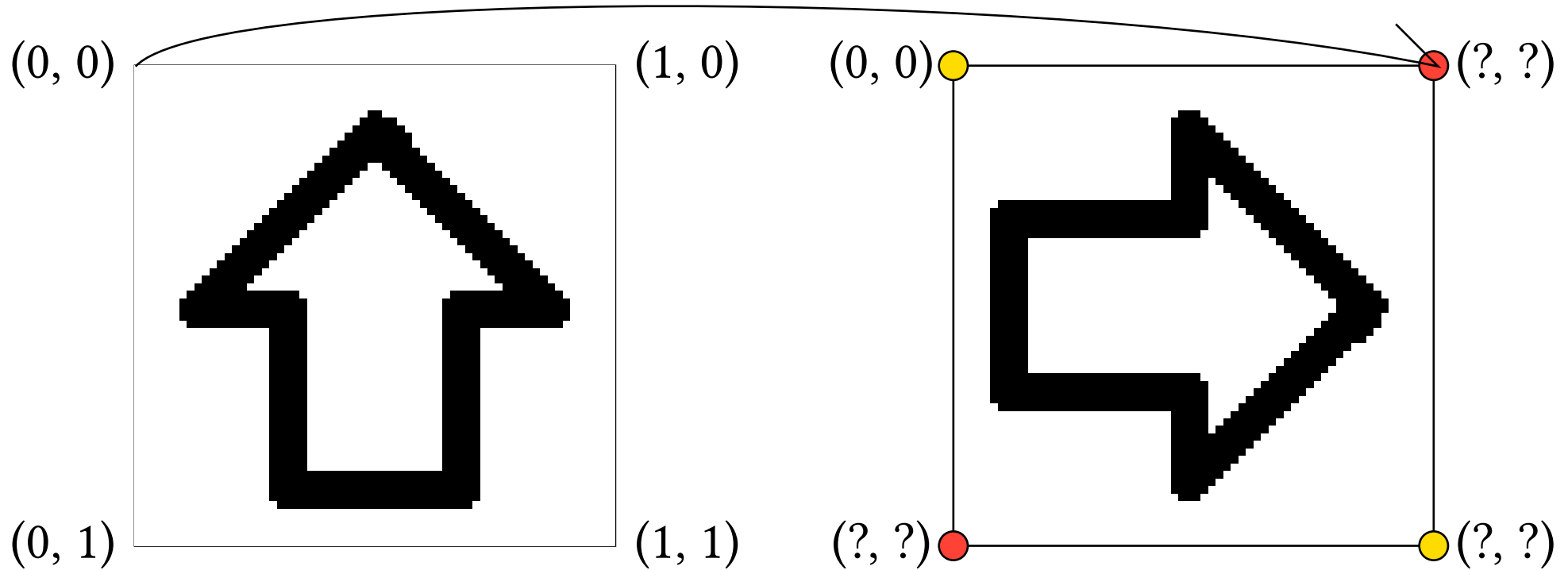


The texture, with the corner UV coordinates labelled. These are always the same for every texture.



Now we want the arrow to be pointing right.

Right arrow mapping answer

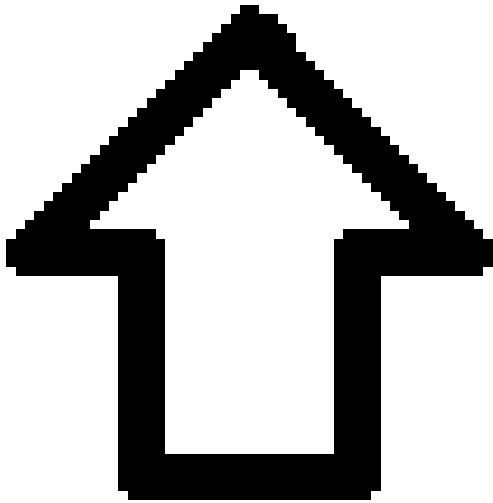


The texture, with the corner UV coordinates labelled. These are always the same for every texture.

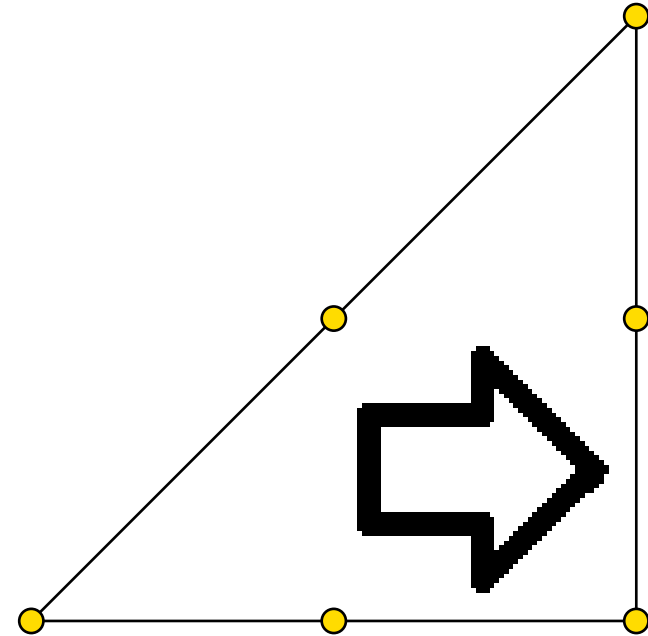
Now we want the arrow to be pointing right.

Our goal

Now let's suppose we want to map this arrow so that it is facing right, and centered within our right triangle. We can add vertices if we want.



What we have



What we want

One solution

One way to do this is to keep the original triangle, but add 6 more vertices